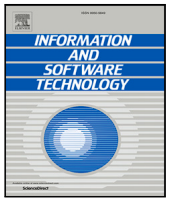
Information and Software Technology 176 (2024) 107565



Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof





A3Test: Assertion-Augmented Automated Test case generation

Saranya Alagarsamy, Chakkrit Tantithamthavorn *, Aldeida Aleti

Monash University, Australia

ARTICLE INFO

Keywords:
Test case generation
Deep learning

ABSTRACT

Context: Test case generation is a critical yet challenging task in software development. Recently, AthenaTest – a Deep Learning (DL) approach for generating unit test cases has been proposed. However, our revisiting study reveals that AthenaTest can generate less than one-fifth of the test cases correctly, due to a lack of assertion knowledge and test signature verification.

Objective: This paper introduces A3Test, a novel DL-based approach to the generation of test cases, enhanced with assertion knowledge and a mechanism to verify consistency of the name and signatures of the tests. A3Test aims to adapt domain knowledge from assertion generation to test case generation.

Method: A3Test employs domain adaptation principles and introduces a verification approach to name consistency and test signatures. We evaluate its effectiveness using 5,278 focal methods from the Defects4j dataset.

Results: Our findings indicate that A3Test outperforms AthenaTest and ChatUniTest. A3Test generates 2.16% to 395.43% more correct test cases, achieves 2.17% to 34.29% higher method coverage, and 25.64% higher line coverage. A3Test achieves 2.13% to 12.20% higher branch coverage, 2.22% to 12.20% higher mutation scores, and 2.44% to 55.56% more correct assertions compared to both ChatUniTest and AthenaTest respectively for one iteration. When generating multiple test cases per method A3Test still shows improvements and comparable efficacy to ChatUnitTest. A survey of developers reveals that the majority of the participants 70.51% agree that test cases generated by A3Test are more readable than those generated by EvoSuite.

Conclusions: A3Test significantly enhances test case generation through its incorporation of assertion knowledge and test signature verification, contributing to the generation of correct test cases.

1. Introduction

Unit testing is a critical component of software development to ensure the quality of software systems and improve developers' productivity [1]. However, writing high-quality and effective test case is a difficult and time-consuming task. Thus, various automated test case generation approaches are proposed. For example, random-based test case generation [2], and search-based test case generation [3]. Previous studies found that random-based and search-based approaches may achieve good code coverage, but do not produce human-readable test cases (e.g., generating a test method name as `test0()`, instead of `testAddition()`). Such human readability is an important test case characteristic that is desirable in practice [4,5]. A lack of human-readable test cases could impact the understanding, debugging, and maintaining activities of the test cases.

Recently, Tufano et al. proposed AthenaTest, a Transformer-based model that is learnt from developer-written test cases in order to generate correct and readable tests [6]. AthenaTest is represented as a translation task where the source is *a focal method* (that is, the method we would like to test), and the target is *the corresponding test case* originally written by a software developer. The test cases generated by AthenaTest are realistic they resemble developer-written test case and are human-readable code with appropriate variable and method names. While AthenaTest demonstrates significant strengths, they found that 16.21% of the generated test cases are correct, meaning that only 16.21% of the generated test cases are syntactically correct, compilable, and correctly asserting the expected behavior of a focal method. This finding highlights that there are areas for future exploration and improvement, such as enhancing assertion knowledge, improving consistency in naming conventions, and verifying test signatures.

Due to the unavailability of the AthenaTest replication package, we first perform a replication study (RS) of Tufano et al. [6] using Defects4j [7] as an evaluation dataset. We first re-implement AthenaTest [6] using the hyperparameter settings reported in the original paper. We find that our implemented AthenaTest does not generate any correct test cases (0%). This could be related to missing details about the parameter settings (e.g., batch size, fine-tuning, epoch). On the

* Corresponding author.
E-mail address: chakkrit@monash.edu (C. Tantithamthavorn).

https://doi.org/10.1016/j.infsof.2024.107565
Received 28 January 2024; Received in revised form 21 August 2024; Accepted 22 August 2024
Available online 30 August 2024
0950-5849/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

other hand, with our modification, we can successfully implement the AthenaTest approach that achieves results with 18.08% being correct, which is similar to the original paper [6] (16.21%). However, the accuracy of the generated test cases is still far from perfect, highlighting the need for further improvement of DL-based test case generation approaches.

Limitation 1: Limited assertion knowledge. Assertions play an important role in unit testing to assess the expected behavior. However, the pre-training process of AthenaTest is only focused on the natural language dataset and source code, with limited assertion knowledge. This limitation is also highlighted by Tufano et al. [6], as they found that 26.71% of the AthenaTest-generated test cases have incorrect assertions.

Limitation 2: Lack of naming consistency and test signatures verification. AthenaTest leverages a general beam search method to generate test cases. However, 9.49% of the test cases generated by AthenaTest are syntactically incorrect. For example, AthenaTest may generate an incorrect test method name as `read()`, where the actual test method name is `testRead()`. In addition, AthenaTest generates incorrect test signatures as `@Test void isLenient()`, where the `public` keyword is missing from the test method.

Therefore, the generated test cases may be syntactically incorrect, incompatible (i.e., a test case that cannot be executed or evaluated correctly due to differences in platform, dependencies, data, configuration and tools) and non-readable test cases, which could impact developers' productivity and impact the cost of software testing.

In this paper, we propose A3Test (Assertion Augmented Automated Test Case Generation), a DL-based test case generation approach that is augmented by assertion knowledge with a mechanism to verify naming consistency and test signatures in order to address the aforementioned gaps of AthenaTest. First, A3Test leverages the domain adaptation principles where the goal is to adapt existing knowledge from an assertion generation task to our test case generation task. To do so, we first build a pre-trained language model of assertions in a self-supervised manner using PLBART architecture [8] with a masked language model. The resulting pre-trained language model has a stronger foundation knowledge of assertions than AthenaTest. The pre-trained language model is fine-tuned for the test case generation task where the objective is to learn the relationship between the focal method and the corresponding test case. For any generated test case, we introduce a verification approach to check the naming consistency (i.e., revising the test method name to be consistent with the focal method name) and the test signatures (i.e., adding missing keywords like `public`, `void`, or `@test` annotations). Finally, we evaluate A3Test using the Defects4j dataset [7], which consists of 5K test methods that span across five large-scale open-source software projects (i.e., Lang, Chart, Cli, Csv, Gson) and compare with the performance of AthenaTest [6], ChatUniTest [9], CodeT5 [10], CodeBERT [11], CodeGPT [12], and PLBART [8] in order to answer the following five research questions.

(RQ1) How effective is A3Test compared to AthenaTest and ChatUniTest?
Results. A3Test generates 25.16% to 395.43% more correct test cases, attains 2.17% to 34.29% higher method coverage, and 25.64% greater line coverage. Additionally, it provides 2.13% to 12.20% more branch coverage, 2.22% to 12.20% better mutation scores, and 2.44% to 55.56% more accurate assertions compared to both ChatUniTest and AthenaTest, respectively in one iteration (i.e., generating one test case per method). Furthermore, A3Test demonstrates significant improvements over AthenaTest and ChatUniTest in multiple iterations. It generates 33%–405% more correct test cases, achieves 35% higher method coverage than AthenaTest (though 15% lower than ChatUniTest), and provides 35% higher line coverage than AthenaTest. In terms of correct assertions, A3Test shows 24%–89% improvement, and it achieves 21% better mutation scores compared to AthenaTest, while achieving same 46% mutation score compared to ChatUniTest for multiple iterations (i.e., generating multiple test cases per method). Overall, A3Test consistently outperforms AthenaTest and is competitive with ChatUniTest, particularly excelling in better performance with test case generation in one iteration.

(RQ2) Does A3Test outperform existing pre-trained models?
Results. Not all pre-trained models are effective in the test case generation task. Nevertheless, A3Test still outperforms the existing pre-trained models (PLBART, CodeGPT, CodeBERT, CodeT5) for the test case generation task.

(RQ3) What is the effect of the assert pre-training and verification components on the performance of A3Test?
Results. The assertion component makes up 38%, and the verification component contributes 23.7% to the relative improvement compared to the basic PLBART model. However, it is important to note that the combined performance of both assertion and verification components is the most effective.

(RQ4) How efficient is A3Test compared to AthenaTest and ChatUniTest?
Results. A3Test takes 2.9 h to generate test cases in one attempt, which is 20.7% faster while being more accurate than AthenaTest. ChatUniTest incurs a total cost of $87.20 for generating test cases across Defects4j projects.

(RQ5) How does the readability of test cases generated by A3Test compare to EvoSuite?
Results. The majority of the participants (70.51%) agree that test cases generated by A3Test are more readable than Evosuite.

Our results lead us to conclude that the augmentation of the assertion knowledges can significantly improve the performance of DL-based automated test case generation to generate test cases that are more syntactically correct, compilable, and correctly asserting the expected behavior of focal methods than AthenaTest [6], ChatUniTest [9], CodeT5 [10], CodeBERT [11], CodeGPT [12], and PLBART [8], while being more readable than Evosuite.

Novelty & Contributions. To the best of our knowledge, the novelty and contributions of this paper are as follows:

- We introduce an assertion-augmented automated test case generation approach A3Test, leveraging the domain adaptation principles.
- An extensive evaluation by comparing our A3Test with six baseline approaches.
- We make the replication package of both A3Test and AthenaTest (the baseline approach which has been replicated) available to the software engineering community for further research.
- A complete ablation study to investigate each step of our A3Test approach.

Open Science. To support the open science community, we publish the studied dataset, scripts (i.e., data processing, model training, and model evaluation) and experimental results in a GitHub repository (https://github.com/A3Test-TestCaseGeneration/A3Test).

2. Background & related work

Software testing is recognized as a critical, complex, and expensive phase in the software development lifecycle. Companies strive to accelerate their software delivery timelines while maintaining high quality and accuracy. To achieve this, they often depend on automated software testing processes, where automated test case generation plays a crucial role. Unit testing serves as the foundational element, followed by the integration and end-to-end testing. The initial testing

efforts should focus on the unit test level, where individual software components (for example, a single method) are tested in isolation to confirm that they function as expected. By focusing on unit testing first, developers build a solid foundation for their software before moving on to more complex integration and end-to-end testing.

2.1. Unit testing in a nutshell

Unit testing is a type of software testing where individual units or components of the software are tested. To perform unit testing, a developer writes a piece of code (unit tests) to verify the code to be tested (unit). The code 1 shows an example of a unit test for the *Calculator* class with a single *method* (i.e., Focal Method) and the corresponding test method (i.e., that verifies the method's behavior with assertions).

There are software tools and frameworks to support writing and running unit tests, such as Junit [13], TestNG [14]. JUnit provides library such as Mockito [15] and assertions to help developers check conditions, outputs, or states in a software program and assess its expected behavior.

```
// Description
The method takes two double values as input
and returns their sum as a double.
The sum is calculated by simply adding the
two input values together using the + operator.
// Focal Class
public class Calculator{
    // Focal Method
    public double Sum(double first, double second){
        return first + second;
    }
    // Test Method/Test Case
    @Test
    public void testSum()
    {
        double first = 10;
        double second = 20;
        var calculator = new Calculator();
        double result = calculator.Sum(first, second);
        assertEquals(30, result); // Correct Assert
             Statement
    }
```

Listing 1: Focal Class Focal Method and Test Case with Assertions

2.2. Automated test case generation

In the past few years, researchers proposed automated test case generation approaches such as EvoSuite [3], Randoop [2], and Agitar [16] based on different techniques, e.g., random-based approaches, search-based approaches and deep learning-based approaches.

Random-based test case generation approaches like Randoop [2] are proposed to automatically generate test cases randomly. Randoop uses a feedback-directed random test generation algorithm. It starts by generating a sequence of method calls and this sequence is generated randomly. Randoop uses feedback from previous tests to guide the selection process. For example, Randoop may be more likely to generate a sequence of method calls that includes methods that have not been tested in the past. It observes the results of the execution to determine whether the program behaved as expected and creates test cases based on the results of the execution.

Search-based test case generation approaches like Evosuite [3] uses an evolutionary algorithm to generate test cases that maximize code coverage. It works by first generating a random population of test cases. Then, it executes the test cases and measures the code coverage. The test cases that achieve the highest code coverage are selected to be the parents of the next generation of test cases. EvoSuite mutates the parents to create the next generation of test cases. Mutations are designed to increase the code coverage of the test cases. It repeats this process until it reaches a time limit or until it finds a set of test cases that achieve a certain level of code coverage.

A major weakness and criticism of these approaches is related to the poor readability and understandability of the generated test cases [17,18] which clearly appear as machine-generated code (i.e., that is created automatically by software tools or algorithms). For example, labels (i.e., names or identifiers assigned to test methods) test case as "test0", "test1". These names provide no information about the content of the tests, and it is impossible to easily locate or access the relevant tests in a codebase by name. Other studies have highlighted different limitations of these automation tools, such as unsatisfactory code quality [19–21] and the inability to adequately meet the software testing needs of industrial developers [4]. These limitations stem from the fact that these approaches (random-based and search-based testing approaches) mainly focus on code coverage as unique objective. They ignore other critical factors crucial to developers [5], including functional correctness, performance, usability, security, error handling, scalability, maintainability, compatibility, regression testing, documentation, and user acceptance.

Deep Learning-based Test Case Generation Recently, AthenaTest [6], a DL-based approach that leverages a sequence-to-sequence BART [22] transformer model to automatically generate test cases by learning from real-world focal methods and developer-written test cases. The transformer paradigm, on which AthenaTest is built, seeks to learn the best practices for generating understandable and readable test cases from developer-written test cases.

Transformer-based language models are typically developed in two stages: pre-training and fine-tuning. AthenaTest is pre-trained on English [23] and Java code [24] and fine-tuned on the methods2test [25] dataset. The AthenaTest model takes a *focal method as input* and generates a *test case as output*. The test cases generated by AthenaTest are (i) *realistic:* they resemble developer-written test case; (ii) *accurate:* they accurately assert the intended behavior of a focal method, and (iii) *human-readable:* they have readable and understandable code with appropriate variable and method names. However, AthenaTest produces a low percentage (i.e., 16%) of correct test cases which emphasizes the need for further improved performance of DL-based test case generation.

3. A revisiting study of AthenaTest

In this section, we revisit the AthenaTest approach, proposed by Tufano et al. [6]. We employ the Methods2Test [25] dataset to conduct the study. We implemented the approach using the parameter settings reported in the original paper. For the English pre-training phase, the BART is trained for 40 epochs using 160 GB of text data sourced from books, Wikipedia, and news articles [23]. Regarding the code pre-training, the BART is trained for 10 epochs using the 239 test Java repositories available in the CodeSearchNet [24]. We used a context size of 512 tokens for the BART model, which is often used due to the computational limitations and memory constraints associated with processing longer sequences. We used the BartTokenizer from the transformers library, trained directly in the code snippet. During the fine-tuning process of AthenaTest we employ the Methods2Test [25] dataset focal context details. The focal context includes the source code of the focal method, the focal class name, constructor signatures, other public method signatures, and public fields represented by fm+fc+c+m+f. The training time for the entire process, including both pre-training and fine-tuning, was approximately 16.15 h. We use the Defects4j [7] as an evaluation dataset and evaluate the model using the number of correct test cases, i.e., the test case that passes the execution and invokes the given focal method.

Table 1
The hyperparameter settings that are reported by AthenaTest [6] and the four additional settings (under a horizontal line) that we modified in order to achieve comparable accuracy.

Hyper-parameters	AthenaTest	AthenaTest (Modified version)
Encoder, Decoder layers	12	12
Learning rate	0.0001	0.0001
Optimiser	Adam	Adam
Code Pre-training Mask	20%	20%
DataSet	Methods2test	Methods2test
Code Pre-Training Epochs	10	10
Eval Beams	No info	4
Batch Size	No info	32
English Pre-Training	40	8
Fine Tuning Epochs	No info	20

Finding 1. Based on our replication, we find that AthenaTest fails to generate any correct test cases (0%) (see Table 2), indicating that we are not able to produce the results as reported in the paper. We perform a manual analysis of the generated test cases by the AthenaTest approach. In our replication, we find that the generated test cases are heavily towards natural languages (e.g., `public class Salim Mehajer ...`), rather than the test cases that should be generated (e.g., `@Test public void testRead(){...}`). We suspect that this should be due to the impact of the pre-training epochs between NL and code corpora. We observe that, for AthenaTest, the Java pre-training uses 10 epochs, while the English pre-training uses 40 epochs. Thus, it could be possible that the pre-training model is learned more towards natural languages than programming languages, generating incorrect test cases towards natural languages. In addition, we also observe that there are three additional settings (e.g., batch size, fine-tuning and beam search) that are not reported in the paper (see Table 1).

Finding 2. We can successfully implement the AthenaTest approach that achieves results (i.e., 18.08%) similar to the original paper [6] (i.e., 16.21%). Since we find that the generated test cases are biased towards natural languages, we reduce the number of epochs for the English pre-training to 8 epochs, which is lower than the Java pre-training to ensure that the model has more influence towards generating Java test cases than generating English text. Table 1 also reports other parameter settings that we used to achieve comparable results.

Limitation. For the replication study, we did not systematically perform parameter tuning, but followed the recommendations from the original study and trialed a few options of possible parameters. The results of our replication can be further improved through parameter tuning. Nevertheless, as shown in Table 2, our modification performs better than the original AthenaTest by 1.8%, ensuring that the comparison between our proposed approach and the AthenaTest as a baseline is fair.

4. A3Test - Assertion augmented automated test case generation

Motivating Scenario. Let us consider Bob, a developer in a large-scale software project that follows rigorous testing practices. Bob's primary responsibility is to test the focal method by writing unit tests. However, manually creating correct unit tests for each focal method is time-consuming and challenging. Bob recently started using a Deep Learning (DL) based approach for generating unit test cases. While the DL-based approach has streamlined some of his work, he notices that it does not often generates correct test cases (i.e., syntactically correct and testing the focal method). Consequently, Bob still spends a significant amount of time reviewing and correcting these test cases.

Therefore, we propose **A3Test**, which is a PLBART-based test case generation approach that is augmented by assertion knowledge with a mechanism to verify naming consistency and test signatures. A3Test leverages the domain adaptation principles where the goal is to adapt the existing knowledge from an assertion generation task to the test case generation task. To do so, we first build a pre-trained language model of assertions in a self-supervision manner using a PLBART architecture [8] with a masked language objective.

The pre-training learning objective is described as predicting the masked tokens of a given focal method and its corresponding assert statements. During the pre-training phase, the language model is trained to fill in the gaps or predict the missing parts of the code, specifically the masked tokens related to focal methods and their associated assert statements. By pre-training the language model on this task, it is likely to develop a stronger foundation knowledge of assertions compared to AthenaTest [6]. The language model learns the patterns, relationships, and contextual understanding of focal methods and their corresponding assert statements through the pre-training process. Then, our pre-trained language model is fine-tuned with the test case generation task where the objective is to learn the relationship between focal methods and the corresponding test cases. For any generated test cases, we introduce a verification approach to check the naming consistency (i.e., revising the test method name to be consistent with the focal method name) and the test signatures (i.e., adding missing keywords like `public`, `void`, or `@test` annotations). Below, we describe the details of our approach (Fig. 1).

4.1. Learning meaningful assert statements

Assertion statements are used to assess the expected behavior of a unit function. However, generating test cases is a difficult task since it involves Testing APIs like assertions, which are more than general knowledge of programming languages. Unfortunately, AthenaTest only builds a pre-trained model based on natural language and source code without considering assertion knowledge.

To address this challenge, we leverage a domain adaptation principle where the goal is to improve the performance of a model on a target domain (i.e., generated test cases) containing insufficient assertion statements by using the knowledge learned by the model from another related domain with adequate labeled data (i.e., generated assertions). Thus, we define the source domain as an assertion generation (Section 4.1), where the target domain is a test case generation (Section 4.2). The pre-trained assert model is built in a self-supervision manner with a Masked Language Model (MLM) objective. The pre-training is performed, with the objective to reconstruct the original data from corrupted data and masking 20% of all tokens. MLM (Masked Language Modeling) trains a model to predict a random sample of input tokens that have been replaced by a `[MASK]` placeholder in a multi-class setting over the entire vocabulary. The special token `[MASK]` is replaced for a random sample of the tokens in the input sequence. Different from Athena2Test which uses a BART architecture, we use the PLBART architecture [8] as a base architecture for building a pre-trained model. The PLBART architecture leverages a bidirectional attention mechanism to capture context from both the past and future in a sequence, which is different from BART which leverages a standard attention mechanism, allowing our A3Test to efficiently learn the sequence of inputs in parallel. In addition, PLBART has been pre-trained on a corpus of natural languages and 7 programming languages, while AthenaTest build a BART pre-trained model on a corpus of natural languages and only one programming language. To build our pre-trained assert model, we use an Atlas dataset [26], which is a large corpus of 188,154 pairs of focal methods and assert statements.

4.2. Learning meaningful test cases

Following the domain adaptation principle, we will transfer the pre-trained assert model to fine-tune it in order to learn meaningful test

Table 2
The percentage of correct test cases of AthenaTest from the original paper [6], our replication, and our modification.

Model	GSON	CLI	CSV	CHART	LANG	Total test cases
AthenaTest [6]	2.89%	11.07%	8.98%	11.7%	23.35%	16.21%
Our Replication	0%	0%	0%	0%	0%	0%
Our Modification(30 Iterations)	3.09%	12.3%	8.83%	12.5%	24.3%	**18.08%**

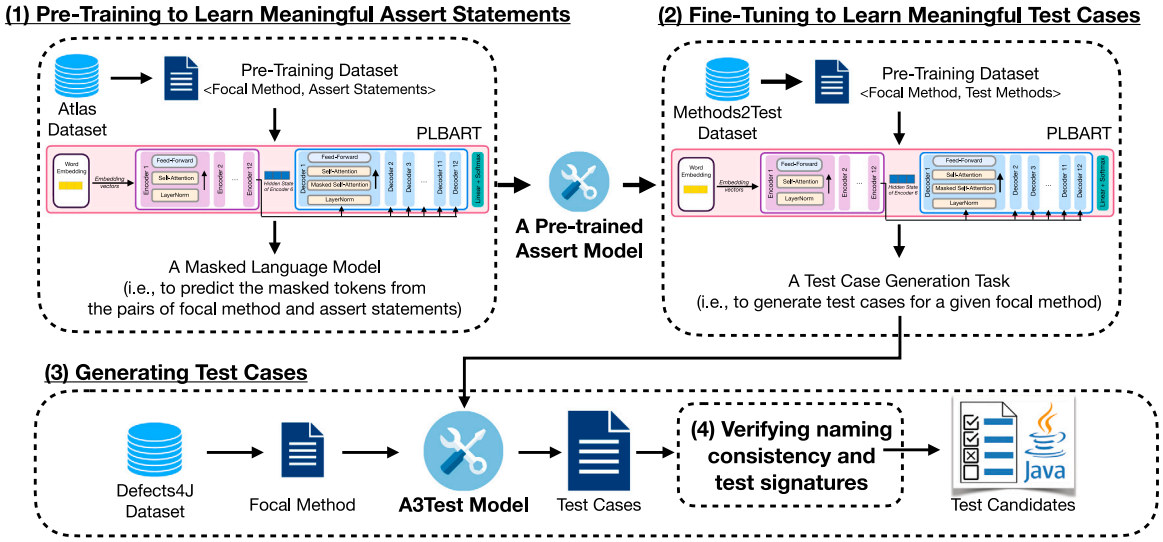


Fig. 1. An overview of our A3Test approach, which is a PLBART-based test case generation approach that is augmented by assertion knowledge with a mechanism to verify naming consistency and test signatures.

Algorithm 1 Beam Search

```
Initialize:
Set beam_width to the desired number of paths to keep
Create initial node with the start symbol, add to active_nodes
repeat
    for each node in active_nodes do
        Expand node to generate all possible successors
        Score each successor using the model's probability
        Add successors to candidates
    end for
    Sort candidates by score in descending order
    active_nodes ← top beam_width candidates
    if all nodes in active_nodes end with end symbol then
        break
    end if
until maximum number of steps reached or all nodes end
return path from node with the highest score in active_nodes
```

cases. To do so, we build an A3Test model by fine-tuning the pre-trained assert model with a test case generation task. The test case generation task is represented as a translation task, with the source being a focal method (i.e., the method we want to test) and the target is the corresponding test case originally written by a software developer. In particular, we fine-tune our A3Test model using the Methods2Test dataset [25]. The Methods2Test dataset consists of 780k pairs of a focal method and the corresponding test methods. We split the dataset into two sets, i.e., training set (80% - 623,958 pairs) and validation set (10% - 78,534 pairs).

4.3. Generating test cases

In the inference phase, we generate a test case for each focal method. To do so, we use a beam search [27] as a decoding method. We leverage beam search to select multiple candidates for an input sequence at each time-step. Algorithm 1 presents the pseudocode of the beam search algorithm used by A3Test. Instead of predicting the token with the highest probability at each time step, beam search explores different parts of the search space simultaneously. The beam search decoding method generates test case candidates while tracking the top-k highest probable candidates (with k being the beam width).

A3Test focuses on generating a single best test case instead of a set of candidates. It employs a best-first search strategy within the beam search framework. This means that instead of generating multiple test cases and selecting the best among them, A3Test prioritizes the generation of the most promising test case at each step. The objective is to find the test case with the highest probability of the correct test case. This allows the beam search to select the best candidates with the highest probability using a best-first search strategy for generating test cases, allowing our A3Test to efficiently generate a single best test case instead of generating 30 candidates like AthenaTest.

4.4. Verifying naming consistency and test signatures

It is possible that the generated test cases could be syntactically incorrect (e.g., generating incomplete parenthesis, incorrect test method names, or invalid test signature keywords), which impacts the performance of our A3Test. To address this challenge, we introduce an automated verification approach (which was not previously done by AthenaTest) in order to check and correct the naming consistency and invalid test signatures. Our verification approach consists of two parts: (1) verify the incomplete parenthesis, (2) verify the test signatures.

Verifying the incomplete parenthesis. It is possible that the generated test cases may have an incomplete parenthesis (e.g., (), {}). Thus, we develop an approach to detect and correct missing parenthesis using a push-pop algorithm. Such an approach allows us to detect and correct the missing parenthesis.

Verify the test signatures. Test cases will not be executed if they do not match the test method signatures of the JUnit framework. In general, Java methods can be private, public, protected, or package-private. However, test cases are encouraged to be a public method only in order to be executed. Unfortunately, DL-based test case generation approaches are not specifically trained to generate public methods only, but could be others too. In addition, there was a lack of a mechanism to verify the test signatures in order to be successfully executed. In general, test method names adhere to the naming convention of the JUnit framework (e.g., a test method name start with `test`). For example, a test method named `testRead()` is considered to enhance readability and clarity as a conventional practice. We check the prefix of the method names (i.e., check the first four letters whether it contains `test` or not). Our approach will add the prefix `test` to the test method name if the prefix is missing. Thus, based on a given example, the incorrect test method name (`read()`) will be automatically revised to `testRead()`

Thus, we introduce an approach to verify the test signatures based on a string processing approach. For a given test method, we first check the sequence and the existence of the first four tokens (i.e., `@Test public void test[MethodName]{..}`) if the following four specific keywords exist or not, i.e., `@Test`, `public`, `void`, and `test`. For a given test method, we first check the sequence and the existence of the first four specific keywords i.e., `@Test`, `public`, `void`, and `test`. Any keywords that are missing from the sequence of the first four tokens will be automatically added to the generated test methods to ensure that the generated test methods by our A3Test will be successfully executed.

5. Experimental setup

In this section, we present the details of our experimental setup, including datasets, model implementation, model training, hyperparameter settings, evaluation measures and baseline comparison.

Datasets. In our experiment, we use three datasets the Atlas [26] dataset, the Methods2Test [25] dataset and the Defects4j [7] dataset.

The Atlas [26] dataset is a corpus of test case prefixes, corresponding method units, and assertions use for pre-training. Atlas was collected from 9K open-source Java projects on GitHub. which contains a total of 188,157 assertions mined from Java projects. We only use Atlas dataset with the focal method and assertions to augment the assert statement.

The Methods2Test [25] dataset is the largest publicly available parallel corpus of test cases mapped to the corresponding focal methods. This dataset enlists 780K mapped test cases, extracted from 91K open-source Java projects. The dataset is a collection of mapped test cases and focal method pairs, where each test case corresponds to a focal method. The code corpus dataset was extracted from 91K open-source Java projects. The dataset contains 780,944 distinct mapped test case pairs. We use the dataset for fine-tuning, the Methods2Test dataset is the only largest publicly available method and test case pair dataset.

The Defects4j [7] dataset is used as a evaluation dataset. Defects4j provides a collection of open source Java projects in the real world that can be used to evaluate and compare various software testing techniques. Each project is a collection of Java programs, including Java class and the focal method. In particular, we chose the same five Defects4j projects as AthenaTest for test case generation, namely, Apache Commons Lang [28], JFreeChart [29], Apache Common CLI [30], Apache Common CSV [31], Google Gson [32]. Subsequently, we parse the focal classes and extract the list of public method for each project. In summary, we have a total of 5278 focal methods, including 2712 focal methods from Lang, 1328 focal methods from Chart, 645 focal methods from Cli, 373 focal methods from Csv, and 220 focal methods from Gson. Each one of these public methods represents a focal method for which we aim to generate test cases.

Model Implementation. A3Test is built on top of two deep-learning Python libraries, Transformers [33] and PyTorch [34]. The Transformers library provides API access to transformer-based model architectures, while the PyTorch library aids in computation during training. We used the PLBART checkpoint from the "uclanlp/plbart-base" model. We pre-trained the model utilizing the Atlas dataset [26], a corpus consisting of 188,154 pairs of focal methods and corresponding assert statements. We refined our model by fine-tuning it with the Methods2Test dataset [25].

Model Training. The PLBART tokenizer and model pre-trained by Ahmad et al. [8] are obtained from the Transformers library. To generate test cases, we use the methods2test dataset to fine-tune our pre-trained assert model. For the AthenaTest approach, we use all the best hyper-parameters described in Tufano et al. [6]. The experiment is run on one NVIDIA GeForce RTX 3090 GPU with 24 GB memory, an Intel(R) Core(TM) i9-9980XE CPU@3.00 GHz with 36 core processors, and 64G RAM.

Hyper-Parameter Settings for Fine-Tuning. For the model architecture of our A3Test approach, we use the default setting of the PLBART base model (i.e., 12 Transformer Encoder/Decoder blocks, and 12 attention heads). During fine-tuning, the learning rate is set to $1e^{-5}$ with a linear schedule. We use AdamW optimizer [35] to fine-tune our A3Test models to update the model and minimize the loss function.

Evaluation Measures. We evaluate the quality of the generated test cases to ensure their effectiveness based on the following measures [36], (i.e., correct test case, focal method coverage, line coverage, branch coverage, mutation score, and correct assertion). For the coverage measures, we execute the generated test cases against a given focal method using the JUnit framework [13] and the JaCoCo [37] library. We describe the details of the evaluation measures below.

- *Correct Test Cases* measures the percentage of generated test cases that are syntactically correct and test the intended focal method. Ideally, all generated test cases would be correct. However, some test cases are incorrect. Thus, we measure the total number of correct test cases generated, which is derived as follows:

$$Correct\ Test\ Case = \frac{Number\ of\ Correct\ Test\ Cases}{Total\ Number\ of\ Test\ Cases} \times 100\%$$

- *Focal Method Coverage (FM)* measures the percentage of focal methods (methods intended to be tested) covered by at least one of the generated test case. It is possible that one generated test case can test multiple focal methods. Thus, we measure the total number of focal methods that are tested by each of the generated test cases, which is derived as follows:

$$FM\ Coverage = \frac{Number\ of\ FM\ Covered}{Total\ Number\ of\ FM} \times 100\%$$

- *Line Coverage* measures the percentage of lines of code(loc) that are tested by the generated test case. In an optimal scenario, all the lines in the focal method should be tested. However, the generated test cases may not cover all lines of the focal method. Thus, we measure the total number of loc in focal methods that are tested by each of the generated test cases, which is derived as follows:

$$Line\ Coverage = \frac{Number\ of\ Executed\ Lines}{Total\ Number\ of\ Lines} \times 100\%$$

- *Branch Coverage* measures the percentage of branches in the focal method (such as if statements or loops) that are covered by the generated test cases. Thus, we measure the total number of branches in focal methods that are tested by each of the generated test cases, which is derived as follows:

$$Branch\ Coverage = \frac{Number\ of\ Executed\ Branches}{Total\ Number\ of\ Branches} \times 100\%$$

- *Mutation Score* measures the percentage of number of mutants killed in the code. Mutation operators are used to systematically create variants of the program under test, each differing by only a small syntactic change. These operators include Arithmetic Operator Replacement (AOR), Logical Operator Replacement (LOR), Shift Operator Replacement (SOR), Relational Operator Replacement (COR), Increment Operator Replacement (ROR), Unary Operator Replacement (ORU), Literal Value Replacement (LVR), and Statement Deletion (STD). The test cases are executed on each mutant; if a test cases that passes on the original program fails on the mutant, then the mutant is killed. To analyze the mutation score, we use the Defects4j command to measure the mutation score *d4j-mutation [-w work_dir] [-r | [-t single_test]] [-i instrument_classes] [-m mutation_operators_file]* . We applied all mutation operators: AOR, LOR, SOR, COR, ROR, ORU, LVR, STD. A high mutation score indicates that the generated test case is effective in detecting faults introduced by the mutations. Thus, we measure the mutation score of the generated test cases, which is derived as follows:

$$Mutation\ Score = \frac{Number\ of\ Mutants\ Killed}{Total\ Number\ of\ Mutants} \times 100\%$$

- *Correct Assertions* measures the number of correct assertions generated by the correct test cases. Ideally, a generated test case should consist of all correct assertions (i.e., syntactically correct and correctly assert the expected behavior of a focal method). However, it is possible that not all assertions are correct. Thus, we measure the total number of assertions generated for each of the generated test case, which is derived as follows:

$$Correct\ Asserts = \frac{No.\ of\ Correct\ Asserts.}{Total\ No.\ of\ Correct\ Asserts.} \times 100\%$$

Baseline Comparison. The current deep learning-based test case generation approaches, such as AthenaTest and ChatUnitTest, are designed to adopt an iterative strategy where a single focal method is employed iteratively to generate multiple test cases. However, this iterative process introduce redundancy of the generated test cases. This redundancy not only adds computational overhead but also impacts the overall runtime.

```
public PieDataset getDataset() {
    return this.dataset;
}
```

Listing 2: Focal Method

```
@Test
public void testGetDataset() {
    PiePlot plot = new PiePlot();
    assertNull(plot.getDataset());
    plot.setDataset(new DefaultPieDataset());
    assertEquals(new DefaultPieDataset(),
        plot.getDataset());
}
```

Listing 3: Generated Test Case by AthenaTest

For example, considering the focal method getDataset() 2 and the generated test case 3 by AthenaTest, we observed that out of 30 iterations, 10 iterations resulted in the generation of correct test cases. Notably, the same test case was generated in 8 out of 10 instances.

In contrast, our A3Test approach generates test cases in a single iteration for each focal method, effectively mitigating these issues by reducing redundancy. By generating a single test case per focal method, A3Test minimizes computational overhead and ensures effective test case generation approach. This approach reduces the time and resources required for both test generation and execution. To establish a fair comparison with all the baseline approaches, we employ the same number of iterations (one iteration per focal method) as the A3Test approach for test case generation, as generating test cases with multiple iterations incurs additional cost, time and resources. Additionally, we also conduct a comparative evaluation of A3Test across multiple iterations (6 iterations), consistent with the methodology applied to ChatUniTest (6 iterations) as originally set in the paper.

- **AthenaTest:** We use our own implementation of AthenaTest from the revisiting study detailed in Section 3.
- **ChatUniTest:** We first downloaded the replication package of ChatUniTest approach [9]. Then, we executed the source code followed by the instructions to replicate.
- **Pre-trained Models:** We directly use the four pre-trained models CodeT5 [10], CodeBERT [11], CodeGPT [12], and PLBART [8] for test case generation.

6. Experimental results

***RQ1** : How effective is A3Test compared to AthenaTest and ChatUniTest?*

Motivation. The current deep learning-based test case generation approaches, AthenaTest [6] and ChatUnitTest [9], are designed to generate test cases across multiple iterations (i.e., one focal method is used to produce several test cases). However, their effectiveness in generating correct test cases in a single iteration remains unclear. Thus, we formulate this RQ to investigate the effectiveness of A3Test compared to AthenaTest and ChatUnitTest in generating correct test cases in a single iteration and multiple iterations.

Approach. To answer this RQ, we evaluate the effectiveness of our A3Test approach and compare it with AthenaTest and ChatUniTest using the following six evaluations detailed in Section 5.

In our evaluation, we focus only on correct test cases that test the focal methods. To do so, we execute test cases through the JUnit [13] framework with Jacoco [37] in order to obtain the test coverage analysis report. In the report, we will be able to identify (1) the correct test cases, (2) the focal methods and line and branch coverage.

Knowing the number of correct test cases and the focal method coverage allows developers to make a better data-informed decision about whether the approach is effective or not. We compute the relative percentage improvement of each measure (m) between our approach and the baseline(AthenaTest and ChatUniTest) as follows:

$$\frac{m_{\text{A3Test}} - m_{\text{baseline}}}{m_{\text{baseline}}} \times 100\%. \quad (1)$$

Results. A3Test generates 25.16%–395.43% more correct test cases, 2.17%–34.29% higher method coverage, 25.64% more line coverage, 2.13%–12.20% more branch coverage, 2.22%–12.20% better mutation scores, and 2.44%–55.56% more correct assertions compared to ChatUniTest and AthenaTest, respectively for one iteration. When generating multiple test cases per method, A3Test still shows notable improvements. A3Test generates 33%–405% more correct test cases and achieves 35% higher method coverage than AthenaTest (though 15% lower than ChatUniTest). It also achieves 35% higher line coverage than AthenaTest and shows a 24% to 89% improvement in correct assertions. Moreover, A3Test achieves 21% better mutation scores compared to AthenaTest, while achieving the same 46% mutation score compared to ChatUniTest for multiple iterations.

Table 3 presents the number of test cases that are correct, passing, failing, build and syntax error of A3Test, AthenaTest and ChatUniTest for one and multiple iterations. For the number of correct test cases for one iteration, our A3Test achieves 40%, meaning that 40% of the generated test cases are correct. On the other hand, AthenaTest achieves as low as 8%, meaning that only 8% of the generated test cases are correct and ChatUniTest achieves 32% of correct test cases. For the number of correct test cases for multiple iterations A3Test achieves

Table 3
(RQ1) A comparison between A3Test, AthenaTest, and ChatUniTest generated test cases in "Pass", "Fail", "Build Error", "Syntax Error" categories, including the number and percentage of correct test cases.

One Iteration							
Project	Focal method	Model	Correct	Pass	Fail	Build error	Syntax error
LANG	2,712	A3Test	1,344 (50%)	1,603 (59%)	106 (4%)	801 (30%)	202 (7%)
		AthenaTest	233 (9%)	852 (31%)	884 (33%)	648 (24%)	328 (12%)
		ChatUniTest	567 (21%)	1,505 (55%)	371 (14%)	723 (27%)	113 (4%)
CHART	1,328	A3Test	416 (31%)	591 (45%)	91 (7%)	507 (38%)	139 (10%)
		AthenaTest	95 (7%)	155 (12%)	219 (16%)	944 (71%)	10 (1%)
		ChatUniTest	425 (32%)	679 (51%)	127 (10%)	437 (33%)	85 (6%)
CLI	645	A3Test	227 (35%)	391 (61%)	49 (8%)	134 (21%)	71 (11%)
		AthenaTest	51 (8%)	80 (12%)	183 (28%)	352 (55%)	30 (5%)
		ChatUniTest	181 (28%)	370 (57%)	54 (8%)	185 (29%)	36 (6%)
CSV	373	A3Test	96 (26%)	147 (39%)	33 (9%)	128 (34%)	65 (17%)
		AthenaTest	28 (8%)	34 (9%)	72 (19%)	179 (48%)	89 (24%)
		ChatUniTest	118 (32%)	184 (49%)	35 (9%)	130 (35%)	24 (6%)
GSON	220	A3Test	31 (14%)	68 (31%)	39 (18%)	80 (36%)	33 (15%)
		AthenaTest	4 (2%)	6 (3%)	52 (24%)	117 (53%)	45 (20%)
		ChatUniTest	77 (35%)	113 (51%)	28 (13%)	64 (29%)	15 (7%)
Total	5,278	A3Test	**2,114 (40%)**	2,800 (53%)	318 (6%)	1,650 (31%)	510 (10%)
		AthenaTest	411 (8%)	1,127 (21%)	1,410 (27%)	2,244 (43%)	501 (9%)
		ChatUniTest	1,663 (32%)	2,851 (54%)	615 (12%)	1,539 (29%)	273 (5%)
Multiple Iterations							
LANG	2,712	A3Test	1,438 (53%)	1,715 (63%)	113 (4%)	857 (32%)	216 (8%)
		AthenaTest	249 (9%)	912 (34%)	946 (35%)	693 (26%)	351 (13%)
		ChatUniTest	1,019 (38%)	1,505 (55%)	371 (14%)	723 (27%)	113 (4%)
CHART	1,328	A3Test	445 (34%)	632 (48%)	97 (7%)	542 (41%)	149 (11%)
		AthenaTest	102 (8%)	166 (13%)	234 (18%)	1,010 (76%)	11 (1%)
		ChatUniTest	551 (41%)	851 (64%)	121 (9%)	311 (23%)	45 (3%)
CLI	645	A3Test	243 (38%)	418 (65%)	52 (8%)	143 (22%)	76 (12%)
		AthenaTest	55 (9%)	86 (13%)	196 (30%)	377 (58%)	32 (5%)
		ChatUniTest	241 (37%)	421 (65%)	37 (6%)	156 (24%)	31 (5%)
CSV	373	A3Test	103 (28%)	157 (42%)	35 (9%)	137 (37%)	70 (19%)
		AthenaTest	30 (8%)	36 (10%)	77 (21%)	192 (51%)	95 (25%)
		ChatUniTest	192 (51%)	214 (57%)	26 (7%)	121 (32%)	12 (3%)
GSON	220	A3Test	33 (15%)	73 (33%)	42 (19%)	86 (39%)	35 (16%)
		AthenaTest	4 (2%)	6 (3%)	56 (25%)	125 (57%)	48 (22%)
		ChatUniTest	96 (44%)	121 (55%)	22 (10%)	63 (29%)	14 (6%)
Total	5,278	A3Test	**2,262 (43%)**	2,996 (57%)	340 (6%)	1,766 (33%)	546 (10%)
		AthenaTest	440 (8%)	1,206 (23%)	1,509 (29%)	2,401 (45%)	536 (10%)
		ChatUniTest	2,099 (39%)	3,112 (59%)	577 (11%)	1,374 (26%)	215 (4%)

43%, AthenaTest achieves as low as 8%, and ChatUniTest achieves 39% of correct test cases.

This is also consistent with each individual Defects4j project, as we find that our A3Test approach consistently performs 112%–617% better than AthenaTest and ChatUniTest in terms of the number of correct test cases. Results show that 53% of test cases passed successfully (i.e., test cases are syntactically correct and executable but not asserting the expected behavior of a focal method), while 6% failed due to wrong assertions or expected behavior. Furthermore, 31% of the test cases resulted in build errors, and 9% encountered syntax errors for one iteration and 33% of the generated test cases encountered build errors, with 10% resulted in a syntax error for multiple iterations. These figures indicate that a significant portion of the test cases faced challenges, with build errors being the most common issue followed by syntax errors.

Table 4 presents the focal method coverage of A3Test, AthenaTest and ChatUniTest. For the focal method coverage, our A3Test achieves 47%, meaning that the generated test cases by our A3Test can cover 47% of the focal methods. On the other hand, AthenaTest achieves 35%, meaning that the generated test cases by AthenaTest can cover 35% of the focal methods and ChatUniTest can cover 46% of the focal methods for one iteration. Considering multiple iterations, ChatUniTest achieves a coverage of 77% for focal methods. Comparatively, A3Test demonstrates a comparable focal method coverage of 62%, while AthenaTest achieves a coverage of 36%. Despite the slightly lower coverage of the focal method in multiple runs, A3Test performs best in one iteration.

This is also consistent with each individual Defects4j project, as we find that our approach consistently 2%–147% performs better than AthenaTest and ChatUniTest in terms of focal method coverage. A3Test, AthenaTest and ChatUniTest complement each other effectively, both A3Test, AthenaTest and ChatUniTest have a substantial 23% overlap, while A3Test offers improved coverage by addressing 13% of unique focal methods that AthenaTest and ChatUniTest do not cover, AthenaTest provides additional coverage for 2% of focal methods not covered by A3Test. ChatUniTest provides unique coverage for 12% of focal methods not covered by A3Test and AthenaTest for one iteration. A3Test, AthenaTest and ChatUniTest has a overlap of 34%. A3Test covers an additional 27% of focal methods not addressed by the other two models, AthenaTest adds 3% unique coverage, and ChatUniTest uniquely covers 33% of focal methods not covered by A3Test and AthenaTest in multiple iterations.

Table 5 presents the line, branch coverage and focal method coverage of A3Test, AthenaTest and ChatUniTest. A3Test outperforms AthenaTest and ChatUniTest across multiple key metrics. A3Test generates test cases that achieve 12% more focal method coverage, 10% greater line coverage, and 8% better branch coverage compared to AthenaTest, and also outperforms ChatUniTest for one iteration. This is consistently observed across each individual project within the Defects4j benchmark. A3Test and ChatUniTest have the same coverage

Table 4
(RQ1) Detailed focal method (FM) coverage between A3Test, AthenaTest, and ChatUniTest.

One Iteration								
Project	Focal Methods	Methods Covered **by A3Test**	Methods Covered **by AthenaTest**	Methods Covered **by ChatUniTest**	Methods Covered **by all models**	Methods Covered **only by** A3Test	Methods Covered **only by** AthenaTest	Methods Covered **only by** ChatUniTest
LANG	2,712	1,573 (58%)	1,168 (43%)	1,369 (50%)	1,168 (43%)	380 (14%)	2 (0%)	103 (4%)
CHART	1,328	452 (34%)	387 (29%)	509 (38%)	320 (24%)	67 (5%)	29 (2%)	151 (11%)
CLI	645	239 (37%)	117 (18%)	286 (44%)	39 (6%)	78 (12%)	8 (1%)	177 (27%)
CSV	373	142 (38%)	128 (34%)	160 (43%)	105 (28%)	23 (6%)	24 (6%)	56 (15%)
GSON	220	90 (41%)	22 (10%)	109 (50%)	22 (10%)	68 (31%)	0 (0%)	87 (40%)
Total	5,278	**2,496** (47%)	1,822 (35%)	2,433 (46%)	1,206 (33%)	616 (13%)	131 (2%)	642 (12%)
Multiple Iterations								
LANG	2,712	1,867 (69%)	1,226 (45%)	2,211 (82%)	1,285 (47%)	582 (21%)	34 (1%)	1,148 (42%)
CHART	1,328	759 (57%)	406 (31%)	1,025 (77%)	352 (27%)	407 (31%)	105 (8%)	278 (21%)
CLI	645	311 (48%)	123 (19%)	481 (75%)	143 (22%)	168 (26%)	21 (3%)	86 (13%)
CSV	373	219 (59%)	134 (36%)	254 (68%)	107 (29%)	112 (30%)	77 (21%)	42 (11%)
GSON	220	95 (43%)	23 (10%)	119 (54%)	20 (9%)	75 (34%)	17 (8%)	3 (1%)
Total	5,278	3,251 (62%)	1,913 (36%)	**4,079** (77%)	1,818 (34%)	1,433 (27%)	163 (3%)	1,717 (33%)

Table 5
(RQ1) Comprehensive Comparison of Coverage Metrics between A3Test, AthenaTest, and ChatUniTest.

One Iteration												
Project	#LOC	Line Coverage			#Branch	Branch Coverage			#FM	FM Coverage		
		A3Test	AthenaTest	ChatUniTest		A3Test	AthenaTest	ChatUniTest		A3Test	AthenaTest	ChatUniTest
LANG	68,673	37,919 (55%)	31,933 (47%)	37,867 (55%)	25,484	13,728 (54%)	14,234 (56%)	13,759 (54%)	2,712	1,573 (58%)	1,168 (43%)	1,369 (50%)
CHART	24,860	9,649 (39%)	7,343 (30%)	9,344 (38%)	10,972	3,567 (33%)	2,990 (27%)	3,490 (32%)	1,328	452 (34%)	387 (29%)	509 (38%)
CLI	11,180	2,254 (20%)	4,005 (36%)	3,798 (34%)	4,073	1,904 (47%)	1,434 (35%)	2,109 (52%)	645	239 (37%)	286 (44%)	286 (44%)
CSV	4,321	1,837 (43%)	1,190 (28%)	1,765 (41%)	2,439	914 (37%)	892 (37%)	917 (38%)	373	142 (38%)	128 (34%)	160 (43%)
GSON	2,712	1,196 (44%)	1,306 (48%)	2,097 (57%)	1,045	349 (33%)	261 (25%)	327 (31%)	220	90 (41%)	90 (41%)	109 (50%)
Total	111,809	54,606 (49%)	43,726 (39%)	**54,871** (50%)	44,013	**20,602** (47%)	17,921 (41%)	20,453 (46%)	5,278	**2,496** (47%)	1,822 (35%)	2,433 (46%)
Multiple Iterations												
LANG	68,673	52,049 (76%)	26,249 (38%)	51,788 (75%)	25,484	17,839 (70%)	14,523 (57%)	19,925 (78%)	2,712	1,867 (69%)	1,226 (45%)	2,211 (82%)
CHART	24,860	10,460 (42%)	7,314 (29%)	12,501 (50%)	10,972	7,554 (69%)	2,909 (27%)	4,138 (38%)	1,328	759 (57%)	406 (31%)	1,025 (77%)
CLI	11,180	5,087 (46%)	1,853 (17%)	5,203 (47%)	4,073	2,851 (70%)	1,395 (34%)	3,932 (97%)	645	311 (48%)	123 (19%)	481 (75%)
CSV	4,321	2,617 (61%)	978 (23%)	2,418 (56%)	2,439	1,707 (68%)	868 (36%)	2,275 (93%)	373	219 (59%)	134 (36%)	254 (68%)
GSON	2,712	1,439 (53%)	827 (30%)	1,873 (69%)	1,045	732 (70%)	254 (24%)	655 (63%)	220	95 (43%)	23 (10%)	119 (54%)
Total	111,809	**74,562** (67%)	35,457 (32%)	**75,173** (67%)	44,013	30,809 (70%)	17,143 (39%)	**31,925** (73%)	5,278	3,251 (62%)	1,913 (36%)	**4,079** (77%)

of the 67% and AthenaTest achieves 32% line coverage and achieves 31% higher method coverage than AthenaTest (though 3% lower than ChatUniTest) for branch coverage for multiple iterations.

A3Test consistently shows higher line and branch coverage, with 49% and 47% across all projects compared to AthenaTest 39% and 41%, respectively, and ChatUniTest 50% and 46%. Although ChatUniTest matches A3Test in line coverage for one iteration.

Table 6 presents the correct assertions and mutation score for A3Test, AthenaTest and ChatUniTest. A3Test consistently outperforms AthenaTest and ChatUniTest in all projects in terms of the percentage of

Table 6
(RQ1) Detailed Mutation score and Total and Correct assertions generated by A3Test, AthenaTest, and ChatUniTest.

One Iteration										
Project	A3Test		AthenaTest		ChatUniTest		#Mutants	Mutation Score		
	Total Assertions	Correct Assertions (%)	Total Assertions	Correct Assertions (%)	Total Assertions	Correct Assertions (%)	Total	A3Test	AthenaTest	ChatUniTest
Lang	10,563	6,153 (58%)	8,353	3,711 (44%)	10,352	6,040 (58%)	425	231 (54%)	219 (52%)	221 (52%)
Chart	4,023	1,876 (47%)	3,975	1,201 (30%)	3,943	1,638 (42%)	212	94 (44%)	78 (37%)	96 (45%)
Cli	2,335	930 (40%)	1,878	684 (36%)	2,288	911 (40%)	114	44 (39%)	37 (32%)	44 (39%)
Csv	857	324 (38%)	659	242 (37%)	840	318 (38%)	101	33 (33%)	31 (31%)	31 (31%)
Gson	314	100 (32%)	241	55 (23%)	308	98 (32%)	89	27 (30%)	24 (27%)	28 (31%)
Total	18,092	**9,393 (42%)**	15,106	5,893 (27%)	17,730	9,005 (41%)	941	**429 (46%)**	389 (41%)	420 (45%)
Multiple Iterations										
Lang	63,378	32,611 (51%)	25,059	7,051 (28%)	72,464	35,938 (50%)	2,529	1,374 (53%)	646 (26%)	1,348 (53%)
Chart	24,138	9,943 (41%)	11,925	2,282 (19%)	27,601	9,746 (35%)	1,261	559 (47%)	230 (18%)	586 (46%)
Cli	14,010	4,929 (35%)	5,634	1,300 (23%)	16,016	5,420 (34%)	678	262 (39%)	109 (16%)	268 (40%)
Csv	5,142	1,717 (33%)	1,977	460 (23%)	5,880	1,892 (32%)	601	196 (33%)	91 (15%)	189 (31%)
Gson	1,884	530 (28%)	723	105 (15%)	2,156	583 (27%)	530	161 (31%)	71 (13%)	171 (32%)
Total	108,552	**49,783 (46%)**	45,318	11,197 (25%)	124,110	53,580 (43%)	5,599	**2,553 (46%)**	1,148 (21%)	**2,562 (46%)**

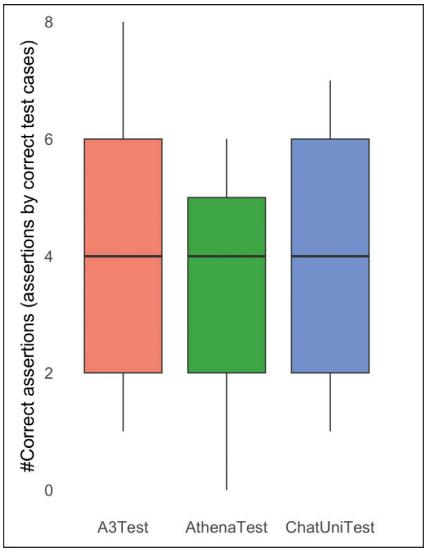


Fig. 2. (RQ1) Distribution of the focal method tested and the correct assertion generated by A3Test, AthenaTest and ChatUniTest.

correct assertions. The overall performance of A3Test is markedly better, achieving 42% ($\frac{9393}{18,092}$) correct assertions compared to 27% ($\frac{5893}{15,106}$) by AthenaTest and 41% ($\frac{9005}{17,730}$) by ChatUniTest. In terms of mutation scores, A3Test achieves higher results 46% across all projects compared to AthenaTest 41% and is on par with ChatUniTest 45%. The data reveals that A3Test has a stronger ability to generate correct assertions for the test cases and better mutation score for one iteration. In comparison with the multiple iterations A3Test achieves higher number of correct assertions with 46% and ChatUnitest achieves 43%. Additionally in terms of mutation scores, A3Test and ChatUniTest achieves higher results 46% across all projects compared to AthenaTest 21%. In detail, across five projects within Defects4j, our A3Test approach consistently outperforms both AthenaTest and ChatUnitTest in the first iteration and, and has comparable performance with ChatUniTest after multiple iterations.

The box plot (Fig. 2) details the distribution of the correct assertions produced by our A3Test, AthenaTest, and ChatUniTest. It demonstrates the distribution of correct assertions (i.e., assertions generated by correct test cases only) for A3Test, AthenaTest, and ChatUniTest across the focal methods. A3Test shows a higher median of approximately 4 and a tighter interquartile range from about 2 to 6, suggesting a higher and more consistent correct assertion count compared to AthenaTest range from 3 to 5 and ChatUniTest performance.

> A3Test significantly outperforms AthenaTest and ChatUniTest in one iteration. Furthermore, A3Test matches or exceeds ChatUniTest in generating test cases efficiently in multiple iterations, demonstrating substantial improvements in all coverage, mutation scores, and correct assertions. A3Test is particularly efficient in generating tests case in single iteration saving cost, time and resources compared to ChatUniTest.

RQ2: Does A3Test outperform existing pre-trained models?

Motivation. The pre-training component plays an important role in test case understanding and generation. However, different test case generation approaches have different pre-training strategies. For example, AthenaTest leverages a BART architecture to build its own pre-trained models via an explicit pre-training strategy. On the other hand, A3Test leverages a PLBART architecture as a base model via an implicit pre-training strategy. Also, there exist other pre-trained language models of code (e.g., CodeT5 [10], CodeBERT [11], CodeGPT [12]). These pre-trained models have been successfully used in various software engineering tasks, e.g., code completion, code summarization, code generation, and code transformation, *but not for test case generation*. Thus, it remains unclear which pre-trained models are the best for test case generation and whether our A3Test outperforms the standard pre-trained models or not. Therefore, we formulate this RQ to investigate the performance of various pre-trained models when compared to our A3Test.

Approach. To answer this RQ, we select the four existing pre-trained models of code, namely, CodeT5 [10], CodeBERT [11], CodeGPT [12], and PLBART [8] as a base model for the test case generation task, without including the other components (Assert+Verification). We perform the fine-tuning on all the pre-trained models using the Methods2Test [25] dataset. Then we compare the performance of these models with our A3Test. Finally, we evaluate the performance of the models using the number of correct test cases.

```
// Focal Method
public static float toFloat(final String str)
 {
 return toFloat(str, 0.0f);
}
-----------------------------------------------
// PLBART
```

```
@Test public void testToFloat() {
     assertEquals(Float.POSITIVE_INFINITY,
     NumberUtils.toFloat("0.0"));
     assertEquals(Float.POSITIVE_INFINITY,
     NumberUtils.toFloat("0.0"));
     assertEquals(Float.POSITIVE_INFINITY,
     NumberUtils.toFloat("0.0"));
     assertEquals(Float.POSITIVE_INFINITY,
-----------------------------------------------
// CodeT5
    @Test public void testToFloat() {
    assertEquals(Float.MAX_VALUE, Float.MAX_VALUE,
         Float.MAX_VALUE); assertEquals(Float.MIN_VALUE,
         Float.MIN_VALUE, Float.MAX_VALUE);
    }
-----------------------------------------------
// CodeBERT
public test float toFloat(final String str) {
    return toFloat(str, 0.0f);}
-----------------------------------------------
// CodeGPT
public float toFloat(final str){ return toFloat(str,
     0.0f); }
-----------------------------------------------
// A3Test
 @Test public void testToFloat()
 {
       assertEquals(2.5f, NumberUtils.toFloat("2.5"),
            0.001f);
       assertEquals(2.5f, NumberUtils.toFloat("2.5"),
            0.001f);
       }
```

Listing 4: Example of test cases generated by the approaches considered in RQ2.

Results. Not all pre-trained models are effective in the test case generation task. Table 7 presents the performance of A3Test and compares it with different pre-trained language models. We find that the performance of the existing pre-trained models varies greatly from 0% (CodeGPT, CodeBERT) to 21.5% (PLBART) for the test case generation task. This finding indicates that different pre-trained models are task-specific. While pre-trained models have been successfully used in various software engineering tasks, e.g., code completion, code summarization, code generation, and code transformation, they do not imply that they will perform best in the test case generation task. This finding highlights the importance of investigating different pre-trained models prior to adopting them for the downstream task.

Code 4 shows an example of the test case generated by PLBART, CodeT5, CodeGPT, CodeBERT, and A3Test for the 'toFloat' method. The PLBART-generated test case is limited in scope, focusing solely on the input value '0.0,' and checking if the method consistently returns Float. POSITIVEINFINITY for this specific input. However, the test case is incomplete, lacking a closing parenthesis, rendering it non-executable. The CodeT5 test case employs assertions involving the maximum and minimum float values, which deviates from the intended functionality of the 'toFloat' method, making it unsuitable for method evaluation. The CodeBERT and CodeGPT models did not generate any syntactically correct test case. In contrast, A3Test effectively tests the method by converting the string '2.5' to the float value 2.5f. This test case aligns with the actual purpose of the 'toFloat' method, evaluating its functionality with a valid input value.

Our A3Test outperforms the existing pre-trained models (PLBART, CodeGPT, CodeBERT and CodeGPT) for the test case generation task. When comparing A3Test (PLBART + Assert + Verification) with PLBART alone, we find that A3Test performs significantly better than PLBART alone, improving from 21.50% to 40.05%. This confirms that using PLBART alone is not effective enough for the test case generation task.

Table 7
(RQ2) The performance of A3Test, AthenaTest and ChatUniTest when compared to other pre-trained language models (measured by the percentage for the correct test cases).

Model	%Correct	GSON	CLI	CSV	CHART	LANG
A3Test	**40.05%**	14.09%	25.19%	25.73%	31.30%	49.50%
AthenaTest	8.09%	3.09%	8.17%	8.98%	7.38%	13.35%
ChatUniTest	32.67%	35.09%	32.1%	28.12%	32.37%	21.05%
PLBART	21.50%	5.04%	13.70%	10.10%	9.26%	22.40%
CodeT5	13.90%	3.63%	11.00%	8.57%	7.60%	19.28%
CodeBERT	0%	0%	0%	0%	0%	0%
CodeGPT	0%	0%	0%	0%	0%	0%

Table 8
(RQ3) The percentage of correct test cases generated by each variant of A3Test, AthenaTest and ChatUniTest modification.

Model	Correct%	GSON	CLI	CSV	CHART	LANG
A3Test	**40.05%**	**14.09%**	**35.19%**	**25.73%**	**31.30%**	**49.50%**
PLBART+Verification	26.60%	11.40%	12.30%	24.04%	19.60%	45.80%
PLBART+Assertion	29.10%	8.60%	25.19%	19.03%	21.30%	36.70%
PLBART	21.50%	5.04%	13.70%	10.10%	9.26%	22.40%
ChatUniTest	32.67%	35.09%	32.1%	28.12%	32.37%	21.05%
AthenaTest+Verification	14.56%	5.56%	14.71%	16.16%	13.28%	24.03%
AthenaTest	8.09%	3.09%	8.17%	8.98%	7.38%	13.35%

In addition, PLBART alone performs better than AthenaTest (the state-of-the-art approach), improving from 8.09% to 21.50%. While both AthenaTest and PLBART are based on the same BART architecture, their pre-training strategies are different. This finding confirms that PLBART, which leverages the implicit pre-training strategy, performs better than the explicit pre-training strategy used by AthenaTest. This highlights the benefit of our A3Test that leverages PLBART instead of using a basic BART architecture.

Furthermore, ChatUniTest also outperforms AthenaTest with a significant margin, achieving 32.67% correctness compared to AthenaTest 8.09%. While ChatUniTest performs well, A3Test outperforms ChatUniTest, highlighting the effectiveness of our approach in generating correct test cases.

> Not all pre-trained models are effective in the test case generation task. Nevertheless, our A3Test still outperforms the existing pre-trained models (PLBART, CodeGPT, CodeBERT and CodeGPT) for the test case generation task.

RQ3: What is the effect of the assert pre-training and verification components on the performance of A3Test?

Motivation. Our A3Test approach consists of three key components, i.e., PLBART + Assert Pre-Training + Verification. However, little is known about which components contribute the most to the performance of our A3Test. Thus, we formulate this RQ to conduct an ablation study to investigate the performance of the components of our A3Test approach.

Approach. We conduct an ablation study to investigate the performance of the components of our A3Test approach. We extend our experiment to systematically evaluate the following four variants of A3Test by removing the Assertion Pre-Training and Verification components as follows:

- `AthenaTest`: The AthenaTest without the assertion pre-training and verification components with one iteration.
- `AthenaTest+Verification`: The AthenaTest with the verification component, but without the assertion pre-training component with one iteration.
- `ChatUniTest`: The ChatUniTest approach with one iteration.

Table 9
(RQ4) The computational time to generate and execute one test case (on average), to generate test cases in one attempt for all of the 5 studied Defects4j projects.

Model	Test case generation		Test execution with coverage analysis		Cost($) for generating Test cases
	Average Time per Test Case (s)	Time for 1 Attempt (h)	Average Time per Test Case (s)	Time for 1 Attempt (h)	Total cost
A3Test	1.98	2.9	3.07	4.5	–
AthenaTest	2.34	3.5	3.07	4.5	–
ChatUniTest	4.31	6.32	–	–	$87.20

- `PLBART`: The PLBART architecture without the assertion pre-training and verification components.
- `PLBART+Verification`: The PLBART architecture with the verification component, but without the assertion pre-training component.
- `PLBART+Assert`: The PLBART architecture with the assertion pre-training component, but without the verification component.
- `PLBART+Assert+Verification`: Our own A3Test.

Results. Table 8 presents the performance of A3Test, AthenaTest, and compares it with different assert pre-training and verification components. **The assertion component of A3Test contributes 38%, while the verification component contributes 23.7% of the relative improvement when compared to the basic PLBART model.** When comparing AthenaTest and AthenaTest + Assertion, we find that the performance has improved from 18.08% to 20.86%, contributing to 15.3% of the relative improvement, which is not substantial. When comparing PLBART and PLBART + Assertion, we find that the performance has improved from 21.50% to 29.10%, contributing to 38% of the relative improvement. On the other hand, when comparing PLBART and PLBART + Verification, we find that the performance has improved from 21.50% to 26.60%, contributing to 23.7% of the relative improvement. These findings highlight that each of our proposed components contributes to the performance improvement of the A3Test.

Moreover, ChatUniTest also shows a significant improvement over AthenaTest, achieving 32.67% correctness compared to AthenaTest 8.09%. This demonstrates the effectiveness of ChatUniTest approach in generating correct test cases. Nevertheless, **our A3Test approach that considers both assertion and verification components still performs the best.** Although we find that each component can contribute to performance improvement to some extent, when considering both assertion and verification components, the performance is improved from 21.50% to 40.05%, accounting for 86.2% ($\frac{40.05-21.50}{40.05}$) of the improvement, highlighting the importance of our proposed assertion and verification components for test case generation.

> The assertion component of A3Test contributes to 38%, while our verification component contributes to 23.7% of the relative improvement when compared to the basic PLBART model. Nevertheless, considering both assertion and verification components, A3Test performs the best.

RQ4: How efficient is A3Test compared to AthenaTest and ChatUniTest?

Motivation. The efficiency of the test case generation approaches is an important perspective to consider about the adoption of research-driven approaches in practice. Thus, we formulate this RQ to investigate what is the efficiency of our A3Test approach when compared to AthenaTest and ChatUniTest.

Approach. Different environments may produce different execution times, which may impact the efficiency of the test case generation approaches. To ensure a fair comparison, we decided to run AthenaTest (the modification version in Section 3) in our environment, which is the same as where we run our A3Test approach. We also run each approach individually to ensure that the time measurement is accurate. For each focal method, we measure the computational time that it takes for each approach to generate test cases. Finally, we report the average time to generate each test case and the total amount of time to generate test cases for all of the 5278 focal methods and the execution time for each test case.

Once the test cases are generated, they are executed through the JUnit [13] framework with Jacoco [37] in order to obtain a test coverage analysis report. The test case execution setup is detailed in several key steps. Initially, the preparation of the test environment involves setting up the necessary software tools, which includes installing Defects4j, configuring Java, and setting up JUnit. This preparation phase takes approximately one hour. Following this, the integration and configuration step involves integrating candidate test cases into a test class and ensuring that all necessary imports and scaffolding are accurately established, which requires about two hours. Finally, the actual test execution phase lasts about 2.5 h for the 5 Defects4j projects.

Results. **A3Test takes 2.9 h to generate test cases in one attempt, which is 20.7% faster while being more accurate than AthenaTest. ChatUniTest incurs a total cost of $87.20 to generate test cases for Defects4j projects.**

Among the total 5278 test cases (see Table 9), A3Test takes 2.9 h and AthenaTest takes 3.5 h. The average time to generate each test case using A3Test is 1.98 s, while AthenaTest requires an average of 2.34 s.

The overall time required to complete one full cycle of the execution of the test case is approximately 5.5 h, with one hour for the setup of the test environment and 4.5 h for the execution of the test and the analysis of coverage for A3Test. This time includes setting up the environment, integrating and configuring the test cases, executing the tests, and analyzing the coverage reports. This comprehensive timeframe includes one hour allocated for setting up the test environment. Additionally, each test execution cycle, along with the subsequent coverage analysis specifically for AthenaTest, spans approximately around 5.5 h.

ChatUniTest, on the other hand, takes 6.32 h for one attempt. The average time to generate each test case by ChatUniTest is 4.31 s, which is considerably higher than both A3Test and AthenaTest. Additionally, ChatUniTest incurs a cost for generating test cases. The cost for generating test cases using ChatUniTest vary across the projects, with Gson costing $10.45, Cli $12.82, Csv $15.11, Chart $19.75, and Lang $29.07, totaling $87.20 for one iteration and $315 for 6 iterations.

> Our A3Test takes 2.9 hours to generate test cases in one attempt, which is 20.7% faster and more accurate than AthenaTest. ChatUniTest incurs a total cost of $87.20 for generating test cases across Defect4j projects for one iteration and $315 for 6 iterations.

RQ5: How does the readability of test cases generated by A3Test compare to EvoSuite?

Motivation. To understand the readability of the generated test cases, we conducted a user study of the A3Test-generated test cases and compared them to Evosuite. Thus, we formulate this RQ to conduct a user study to investigate the readability of the test cases of our A3Test approach.

Approach. We aim to unveil the readability of the test cases generated by our A3Test approach. We conduct our user study according to the following steps: (1) design and develop a survey, (2) select participants and (3) survey questions. We explain the details of each step below.

(i) Design and Development of the Survey: Our survey utilizes a cross-sectional design, capturing responses from participants at a singular point in time. Comprising three closed-ended and one open-ended question, the survey is structured to be completed anonymously in approximately 20 min. The closed-ended questions use a multiple-choice format, and the survey itself is divided into two sections: initial demographic questions and questions regarding the readability of test cases generated by A3Test and EvoSuite. We employed Qualtrics as the platform for our online survey administration, participants encountered a detailed introductory statement upon entering the survey, which outlines the study's goals, the basis for participant selection, potential benefits and risks, and our dedication to maintaining confidentiality. Importantly, our survey underwent a rigorous evaluation process and received ethical approval from the Monash University Human Research Ethics Committee (MUHREC ID: 38015).

During the survey, participants were presented with sample test cases labeled as TestCaseA and TestCaseB, without explicitly mentioning A3Test or EvoSuite. For the five Defects4j projects we randomly selected two test cases from each project, resulting in a total of 10 test cases. Participants were asked to evaluate the readability of each test case by answering three questions: the first focused on determining which test case was easier to read; the second aimed to identify which test case had test method names that best describes the functionality of the test; the third asked participants to express their preference for one test case over the other. By using generic labels for the test cases and avoiding the explicit naming of A3Test or EvoSuite, we aimed to minimize any potential bias or preconceived notions participants might have had. This approach ensured that their evaluations were based solely on the readability of the test cases rather than any prior perceptions of the test generation tools. (ii) Participant Selection: The target population for our study is Java software practitioners. We reached out to potential participants through LinkedIn and Facebook. Participants were invited to complete the online survey, which assessed the readability of test cases. Finally, we obtained a total of 32 responses in two weeks.

(iii) Survey Questions: The survey begins with a question about the participant's role within their software development team (“(Q1). What is your role in the software development team?”). The next question concerns the years of experience the participants have in Java programming (“(Q2). How many years of experience do you have in Java programming?”), aimed at achieving a diverse range of responses from software practitioners with varied professional experience levels.

The survey included precisely three questions presented to the participants: “(Q1) Which test case is easier to read?”; followed by (Q2) “Which test case has test method names that best describe the functionality of the test?”; finally (Q3) “Which test case do you prefer?”.

To ensure the completeness of survey responses, especially for open-ended questions, we conducted a detailed manual review. Out of a total of 32 responses, we identified 6 invalid responses—due to unanswered open-ended questions or incomprehensible replies—and excluded them from further analysis. Consequently, we included the remaining 26 valid responses for analysis. The closed-ended responses were quantitatively analyzed and are presented using the stacked bar plots.

Results. The survey results revealed that developers mostly preferred A3Test test cases in terms of readability, ‘A3Test’ stands out with the highest average percentage of approximately 70.51%.

We received responses from 26 developers, all of whom had experience with Java and JUnit. The distribution of their experience levels is diverse, providing a broad perspective on the survey topics. Specifically, 7.7%(2) of the participants had less than one year of experience, 30.8%(8) had between 1 to 5 years, another 30.8%(8) had between 6 to 10 years, 11.5%(3) had between 11 to 15 years, and 7.7%(2) had between 16 to 20 years of experience. This range in experience levels suggests that the insights gathered encompass both the fresh perspectives of newer practitioners and the seasoned understanding of experienced developers.

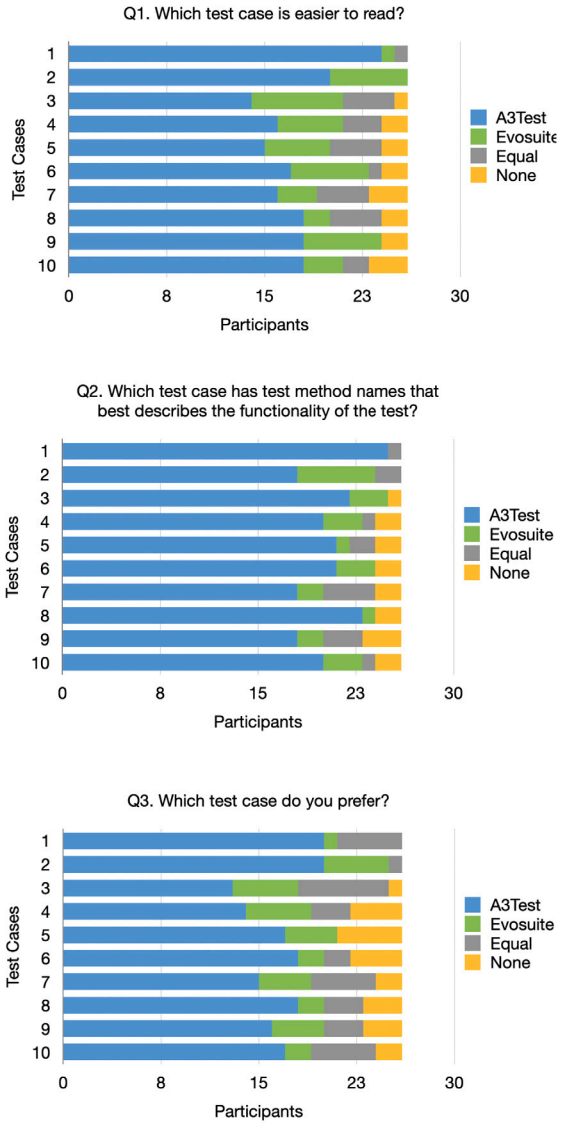


Fig. 3. (RQ5) Survey Results.

Fig. 3 reports the answers to the three survey questions in a stacked bar plots, where the y-axis represents the testing scenario instance, and the x-axis the number of responses for A3Test (in blue, towards left), for EvoSuite (in green, towards right), neutral answer (in grey, towards right) and none answer (yellow). For Q1 a significant majority, 67.69%, found A3Test-generated test cases easier to read, compared to only 16.92% who preferred the EvoSuite test cases. Q2 assess which test case method names best described their functionality. Again, a large majority, 79.23%, favored A3Test, demonstrating a strong preference for the clarity of A3Test's method names. Only 9.23% preferred the method names from EvoSuite. Finally for Q3 participants were asked about their overall preference for a test case based on its alignment with the method's purpose and expected behavior. The results showed that 64.62% preferred A3Test, 14.62% favored EvoSuite, 13.08% gave a neutral response, and 9.23% chose none.

These findings demonstrate a clear preference for A3Test over EvoSuite regarding readability, descriptiveness of method names.

> A3Test's test cases were preferred by developers in terms of readability, with approximately 70.51% finding them readable, while Evosuite had a lower average percentage of developer preference for readability, with approximately 13.59%.

7. Discussion

Implications to Practitioners. The motivation behind developing our A3Test approach is to help practitioners effectively generate test cases. We find that our A3Test approach outperforms all the state-of-the-art approaches (i.e., AthenaTest and ChatUniTest) (RQ1) and other pre-trained models (i.e., PLBART, CodeT5, CodeGPT, and CodeBERT) (RQ2). It is also more cost-effective and faster (RQ4), and more readable (RQ5) than the state-of-the-art approach (i.e., Evosuite).

Traditionally, the existing deep learning-based automated test case generation approaches do not often generate correct test cases (i.e., syntactically correct and testing the focal method). Thus, developers still spends a significant amount of time reviewing and correcting these test cases. Our A3Test approach aims to generate correct test cases by incorporating assertion knowledge and test signature verification. A3Test generates a higher number of correct test cases that are syntactically correct, compilable, and correctly asserting the expected behavior of a focal method. In practice, developers can execute the correct test cases generated by A3Test to test the focal method and detect failures in the software if it is faulty.

8. Threats to validity

As for any empirical study, there are various threats to the validity of our results and conclusions.

Threats to the internal validity related to the degree to which our study minimizes systematic error. Our AthenaTest replication consists of various hyperparameter settings (i.e., number of hidden layers, number of attention heads, and learning rate). Prior studies raise concerns that different hyperparameter settings may have an impact on the evaluation results. However, finding an optimal hyperparameter setting can be very expensive given a large search space of the Transformer architecture. Instead, the goal of our work is not to find the best hyperparameter setting, but to fairly compare the accuracy of our approach with the existing baseline approaches. Thus, the accuracy reported in the paper serves as a lower bound of our approach, which can be even further improved through hyperparameter optimization. To mitigate this threat, we report the hyperparameter settings of our replication package to aid future replication studies.

Threats to external validity concern the generalization of our findings. Experiments are based on five projects from Defects4j [7]. This is in line with the prior study on AthenaTest. We specifically focused on the fixed versions of Defects4j to concentrate on generating correct and reliable test cases. To circumvent the threat to external validity, projects were selected with diversity in mind. The five projects represent different domains of inputs (string, int, etc.) and sizes and complexity of classes. Further experiments with other projects would help with the generalizability of the results.

Threats to construct validity are related to the potential data leakage (i.e., when instances in the training dataset appear in the evaluation set) of our A3Test model. To mitigate this threat, we focus on identifying instances in the training dataset (i.e., Atlas, method2test) that appear in the evaluation set (i.e., Defects4J). For the Atlas dataset, we discovered and removed 52 instances for the CSV project and 9 instances for the GSON project, which appear in the Atlas dataset, and also appear in the Defects4J dataset, prior to the A3Test model pre-training step. Similarly, for the method2test dataset, we discovered and removed 63 instances for the JFreeChart project. This will ensure that there is no data leakage in our experiments. The revised dataset and the updated A3Test model checkpoint is publicly available at the Hugging Face model repository (https://huggingface.co/SaranyaAakas/A3Test).

9. Conclusion

In this paper, we propose A3Test, a pre-trained Transformer-based approach that is augmented by assertion knowledge with a mechanism to verify naming consistency and test signatures. Experiments show that A3Test consistently outperforms AthenaTest and ChatUniTest across all testing metrics in one iteration. While A3Test shows slightly lower performance in focal method coverage and branch coverage over multiple runs, it excels at generating effective test cases in a single iteration, thereby reducing cost, time and resource compared to ChatUnitTest. Specifically, A3Test produces between 25.16% and 395.43% more correct test cases, with method coverage ranging from 2.17% to 34.29% higher, and 25.64% greater line coverage in a single iteration. Additionally, it provides 2.13% to 12.20% more branch coverage, 2.22% to 12.20% better mutation scores, and 2.44% to 55.56% more accurate assertions.

When generating multiple test cases per method, A3Test still shows notable improvements. It generates 33%–405% more correct test cases and achieves 35% higher method coverage than AthenaTest (though 15% lower than ChatUniTest). It also achieves 35% higher line coverage than AthenaTest and shows a 24% to 89% improvement in correct assertions. Moreover, A3Test achieves 21% better mutation scores compared to AthenaTest, while achieving the same 46% mutation score compared to ChatUniTest for multiple iterations. A3Test also surpassed existing pre-trained models such as PLBART, CodeGPT, CodeBERT, and CodeT5 for test case generation. The proposed assertion pre-training and verification components played a significant role in performance improvement. Furthermore, A3Test was much faster than AthenaTest, with a speed improvement of 20.7% while maintaining higher accuracy. ChatUniTest incurs cost for generating test cases. Finally, the survey of developers reveal that the majority of the participants (70.51%) agree that test cases generated by A3Test are more readable than those generated by EvoSuite. In essence, considering all evaluation aspects A3Test is the best approach for effectively generating test cases in one iteration. This is important, because multiple iterations can be expensive, in terms of cost, time and resources.

CRediT authorship contribution statement

Saranya Alagarsamy: Writing – original draft, Methodology, Formal analysis, Data curation, Conceptualization. **Chakkrit Tantithamthavorn:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Funding acquisition, Conceptualization. **Aldeida Aleti:** Writing – review & editing, Writing – original draft, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] M. Cohn, Succeeding with Agile: Software Development Using Scrum, Pearson Education, 2010.

[2] C. Pacheco, M.D. Ernst, Randoop: feedback-directed random testing for java, in: Companion To the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion, 2007, pp. 815–816.

[3] G. Fraser, A. Arcuri, Evosuite: automatic test suite generation for object-oriented software, in: Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering, 2011, pp. 416–419.

[4] M.M. Almasi, H. Hemmati, G. Fraser, A. Arcuri, J. Benefelds, An industrial evaluation of unit test generation: Finding real faults in a financial application, in: 2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP), IEEE, 2017, pp. 263–272.

[5] S. Shamshiri, Automated unit test generation for evolving software, in: Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering, 2015, pp. 1038–1041.

[6] M. Tufano, D. Drain, A. Svyatkovskiy, S.K. Deng, N. Sundaresan, Unit test case generation with transformers and focal context, 2020, arXiv preprint arXiv:2009.05617.

[7] R. Just, D. Jalali, M.D. Ernst, Defects4J: A database of existing faults to enable controlled testing studies for java programs, in: Proceedings of the 2014 International Symposium on Software Testing and Analysis, 2014, pp. 437–440.

[8] W.U. Ahmad, S. Chakraborty, B. Ray, K.-W. Chang, Unified pre-training for program understanding and generation, 2021, arXiv preprint arXiv:2103.06333.

[9] Z. Xie, Y. Chen, C. Zhi, S. Deng, J. Yin, ChatUniTest: a ChatGPT-based automated unit test generation tool, 2023, arXiv preprint arXiv:2305.04764.

[10] Y. Wang, W. Wang, S. Joty, S.C. Hoi, Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, 2021, arXiv preprint arXiv:2109.00859.

[11] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, et al., Codebert: A pre-trained model for programming and natural languages, 2020, arXiv preprint arXiv:2002.08155.

[12] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang, et al., Codexglue: A machine learning benchmark dataset for code understanding and generation, 2021, arXiv preprint arXiv:2102.04664.

[13] JUnit 5, 2018. URL https://junit.org/junit5/.

[14] Testng, 2022. URL https://testng.org/.

[15] Site.mockito.org, 2023. URL https://site.mockito.org/.

[16] agitar, 2022. URL http://www.agitar.com/.

[17] E. Daka, J. Campos, G. Fraser, J. Dorn, W. Weimer, Modeling readability to improve unit tests, in: Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering, 2015, pp. 107–118.

[18] G. Grano, S. Scalabrino, H.C. Gall, R. Oliveto, An empirical investigation on the readability of manual and generated test cases, in: 2018 IEEE/ACM 26th International Conference on Program Comprehension, ICPC, IEEE, 2018, pp. 348–3483.

[19] F. Palomba, D. Di Nucci, A. Panichella, R. Oliveto, A. De Lucia, On the diffusion of test smells in automatically generated test code: An empirical study, in: 2016 IEEE/ACM 9th International Workshop on Search-Based Software Testing, SBST, IEEE, 2016, pp. 5–14.

[20] G. Grano, F. Palomba, D. Di Nucci, A. De Lucia, H.C. Gall, Scented since the beginning: On the diffuseness of test smells in automatically generated test code, J. Syst. Softw. 156 (2019) 312–327.

[21] F. Palomba, A. Panichella, A. Zaidman, R. Oliveto, A. De Lucia, Automatic test case generation: What if test code quality matters? in: Proceedings of the 25th International Symposium on Software Testing and Analysis, 2016, pp. 130–141.

[22] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, L. Zettlemoyer, Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension, 2019, arXiv preprint arXiv:1910.13461.

[23] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, Roberta: A robustly optimized bert pretraining approach, 2019, arXiv preprint arXiv:1907.11692.

[24] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, M. Brockschmidt, Codesearchnet challenge: Evaluating the state of semantic code search, 2019, arXiv preprint arXiv:1909.09436.

[25] M. Tufano, S.K. Deng, N. Sundaresan, A. Svyatkovskiy, Methods2Test: A dataset of focal methods mapped to test cases, in: Proceedings of the 19th International Conference on Mining Software Repositories, 2022, pp. 299–303.

[26] C. Watson, M. Tufano, K. Moran, G. Bavota, D. Poshyvanyk, On learning meaningful assert statements for unit test cases, in: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, 2020, pp. 1398–1409.

[27] V. Raychev, M. Vechev, E. Yahav, Code completion with statistical language models, in: Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation, 2014, pp. 419–428.

[28] Apache Common Lang, 2022. URL https://commons.apache.org/proper/commons-lang/.

[29] JFreeChart, 2022. URL https://jfree.org/jfreechart/.

[30] Apache Commons CLI, 2022. URL https://commons.apache.org/proper/commons-cli/.

[31] Commons CSV, 2022. URL https://commons.apache.org/proper/commons-csv/.

[32] Google Gson, 2019. URL https://github.com/google/gson/.

[33] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, et al., Huggingface's transformers: State-of-the-art natural language processing, 2019, arXiv preprint arXiv:1910.03771.

[34] R. Collobert, K. Kavukcuoglu, C. Farabet, Torch7: A matlab-like environment for machine learning, in: BigLearn, NIPS Workshop, (no. CONF) 2011.

[35] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, 2017, arXiv preprint arXiv:1711.05101.

[36] H. Hemmati, How effective are code coverage criteria? in: 2015 IEEE International Conference on Software Quality, Reliability and Security, IEEE, 2015, pp. 151–156.

[37] Java Code Coverage Library, 2019. URL https://www.eclemma.org/jacoco/trunk/index.html.